SONiX 32-Bit Cortex-M0 Micro-Controller

Timer/Capture/PWM Example Code

AMENDMENT HISTORY

Version	Date	Description
1.0	2024/12/02	1. First version.

Table of Content

1 OVERVIEW

This document provides our customers the C program examples of TimerCapturePWM.

2 FEATURE

- 16-bit and 32-bit Timer setting description.
- Match Registers (MR0~MR3) of the Reset/Stop/Interrupt Enable examples.
- MR0~MR3 and PWM I/O setting examples.
- External Match mode set PWM I/O output operation examples.
- CAP0 input of Counter and Timer Operation example.
- Use ARM-compiler V6.12

3 HW

- The starter kit board of vendor desired MCU.
- Circuit connection will be explained in each Demo Case.

4 EXAMPLE CODE

4.1 Polling Flags

CT16Bx and CT32Bx ".c" and ".h" files will individually declare an interrupt flag variable. The example below is the variable iwCT16B0_IrqEvent declared in CT16B0.h, which is a bit-mask variable mapped to each bit of CT16Bn_RIS.

```
/*_____ D E C L A R A T I O N S ____________________________________________*/
volatile uint32_t iwCT16B0_IrqEvent = 0x00; //The bitmask usage of iwCT16Bn_IrqEvent
```

4.2 Interrupt

Demo Case of IRQ Handler will first determine which IE will be enabled and enable the corresponding interrupt. FW will check the status of the flag variable which corresponds to the interrupt in Main Loop.

```
/*****************************************************************************
 * Function    : CT16B0_IRQHandler
 * Description : ISR of CT16B0 interrupt
 * Input       : None
 * Output      : None
 * Return      : None
 * Note        : None
 *****************************************************************************/
__irq void CT16B0_IRQHandler(void)
{
  uint32_t iwRisStatus;

  iwRisStatus = SN_CT16B0->RIS; //Save the interrupt status.

  //Before checking the status, always re-check the interrupt enable register first.
  //In practice, user might use only one or two timer interrupt source.
  //Ex: Enable only MR0IE and MR3IE ==> No check on MR1IE, MR2IE, and CAP0IE is necessary.
  //User can add the directive pair of "#if 0" and "#endif" pair
  //to COMMENT the un-used parts to reduce ISR overheads and ROM usage.

  //Check the status in oder.
  //MR0
  if (SN_CT16B0->MCTRL_b.MR0IE)       //Check if MR0 IE enables?
  {
    if(iwRisStatus & mskCT16_MR0IF)
    {
      iwCT16B0_IrqEvent |= mskCT16_MR0IF;
      SN_CT16B0->IC = mskCT16_MR0IC;  //Clear MR0 match interrupt status
    }
  }
```

4.3 Main Function

There are several Demo Cases for users to reference. The user can select the sample program to be executed within the blue box below in the main function. Each chip series supports different CT16B functions. Please refer to the TimerCapturePWM sample code for instructions

```
/*****************************************************************************
* Function    : main
* Description : Demo codes of CT timers.
* Input       : None
* Output      : None
* Return      : None
* Note        : None
*****************************************************************************/
int main (void)
{
  //User can configure System Clock with Configuration Wizard in system_SN32F240.c
  SystemInit();

  PFPA_Init();                          //User shall set PFPA if used, do NOT remove!!!

  //-----------------------------------------------------------------------------
  //User Code starts HERE!!!

  //Select the timer demo case
  MN_CtDemoCase0();   //Use CT16B0 to demo MR0 match and stop TC function for 1-ms.
  //MN_CtDemoCase1();   //Use CT16B1 to demo MR1 match and reset TC function for 1-ms period.
  //MN_CtDemoCase2();   //Use CT16B2 to demo MR2 match interrupt first and TC stop by MR3.
  //MN_CtDemoCase3();   //Use CT32B0 to demo PWM function.
  //MN_CtDemoCase4();   //Use CT32B1 to demo PWM waveforms with External Match method.
  //MN_CtDemoCase5();   //Use CT32B2 to demo the Counter function by using P1.0.
  //MN_CtDemoCase6();   //CT16B0/CT32B0: Demo the Capture function(CT16B0) by PWM output(CT32B0).
  while (1);
}
```

4.4 Demo Case 0

This example is implemented with CT16B0. The TC count will match MR0 after about 1ms and stop counting. The user can observe through P1.0 output to know when TC starts counting and when TC stops. Users can observe an approximately 1ms pulse, and the program should stop at the break point below, the result is shown in Section 5.1.

Demo case must first call CT16B0_Init function to enable PCLK, and set MR0 match value to 12000 to make TC count match MR0 after about 1ms. When the setting is done, reset the Timer. After the reset completes, P1.0 is set to high, FW enables TC counting and CT16B0 NVIC interrupt.

In while (1) loop, FW will check whether the MR0 match interrupt is triggered or not, and P1.0 will be set to low and FW will leave the loop if yes. Since the TC counter will stop when it matches the previously setting MR0 match value 12000, the program will lock in a red box below.

```
 91  void MN_CtDemoCase0(void)
 92  {
 93    CT16B0_Init();
 94
 95    SN_CT16B0->MR0 = 12000;   //Set MR0 value for 1ms period ==> count value = 1000*12 = 12000
 96    SN_CT16B0->MCTRL = (mskCT16_MR0STOP_EN|mskCT16_MR0IE_EN); //Set MR0 match as TC stop, and enable MR0 interrupt
 97    SN_CT16B0->TMRCTRL = (mskCT16_CRST|mskCT16_CM_EDGE_UP); //Set CT16B0 as the up-counting mode.
 98    while (SN_CT16B0->TMRCTRL & mskCT16_CRST);  //Wait until timer reset done.
 99    //Use P1.0 to indicate CT16B0 MR0 matches and TC stop.
100    SN_GPIO1->MODE = 0x01;  // Set P1.0 as output.
101    SN_GPIO1->DATA = 0x01;  // Default set P1.0 as output high.
102    SN_CT16B0->TMRCTRL |= mskCT16_CEN_EN; //Let TC start counting.
103    CT16B0_NvicEnable();    //Enable CT16B0's NVIC interrupt.
104    while (1)
105    {
106      if (iwCT16B0_IrqEvent == mskCT16_MR0IF) //Check if MR0 match interrupt occurs
107      {
108        iwCT16B0_IrqEvent = 0;    //Clear MR0 match interrupt variable.
109        SN_GPIO1->DATA = 0x00;    //Set P1.0 as output low.
110        break;
111      }
112    }
113    while (SN_CT16B0->TC == 12000);//Pass: TC should be always equal to 12000.
114    while (1);                    //Fail: Can't run here.
115  }
```

4.5 Demo Case 1

This example is implemented with CT16B1. The TC count will match MR0 after about 1ms and then recount. The user can observe through P1.0 output to know the TC counting period. Users can observe an approximately 500Hz frequency output, the result is shown in Section 5.2.

Demo case must first call CT16B1_Init function to enable PCLK, and set MR1 match value to 12000 to make TC count match MR1 after about 1ms. When the setting is done, reset the Timer. After the reset completes, P1.0 is set to low, FW enables TC counting and CT16B1 NVIC interrupt.

In while (1) loop, FW will check whether the MR1 match interrupt is triggered or not, and P1.0 will toggle if yes. Since the TC counter will recount when it matches the previously setting MR1 match value 12000, MR1 interrupt will be trigger every 1ms, and the frequency of P1.0 output is 500Hz.

```
127  void MN_CtDemoCase1(void)
128  {
129    CT16B1_Init();
130
131    SN_CT16B1->MR1 = 12000;//Set MR1 value for 1ms period ==> count value = 1000*12 = 12000
132    SN_CT16B1->MCTRL = (mskCT16_MR1RST_EN|mskCT16_MR1IE_EN);//Set MR1 match as TC RESET, and enable MR1 interrupt
133    SN_CT16B1->TMRCTRL = (mskCT16_CRST|mskCT16_CM_EDGE_UP);//Set CT16B0 as the up-counting mode.
134    while (SN_CT16B1->TMRCTRL & mskCT16_CRST);//Wait until timer reset done.
135    SN_CT16B1->TMRCTRL |= mskCT16_CEN_EN; //Let TC start counting.
136    CT16B1_NvicEnable();//Enable CT16B1's NVIC interrupt.
137
138    //Use P1.0 toggle when CT16B1 MR1 matches and TC reset.
139    SN_GPIO1->MODE = 0x01;  //Set P1.0 as output.
140    SN_GPIO1->DATA = 0x00;  //Default set P1.0 as output low.
141    while (1)
142    {
143      if (iwCT16B1_IrqEvent == mskCT16_MR1IF) //Check if MR1 match interrupt occurs
144      {
145        iwCT16B1_IrqEvent = 0;    //Clear MR1 match interrupt variable.
146        SN_GPIO1->DATA ^= 0x01;   //Toggle P1.0.
147      }
148    }
149  }
```

4.6 Demo Case 2

This example is implemented with CT16B2. The MR2 match interrupt is triggered when the TC count matches MR2, and stop TC counting when the TC count matches MR3. The user can observe through P1.0 and P1.1 output high pulse to know when MR2 matches TC and when MR3 matches TC. Users can observe that P1.0 outputs a 1ms pulse, and P1.1 outputs a 3ms pulse. The program should be stopped at the breakpoint below, the result is shown in Section 5.3.

Demo case must first call CT16B2_Init function to enable PCLK, and set MR2/MR3 match value to 12000/36000 to make TC count match MR2 after about 1ms, and match MR3 after about 3ms. When the setting is done, reset the Timer. After the reset completes, P1.0 and P1.1 are set to high, FW enables TC counting and CT16B2 NVIC interrupt.

In while (1) loop, FW will check whether the MR2/MR3 match interrupt is triggered or not, P1.0/P1.1 will be set to low individually, and FW will leave the loop if yes. Since the TC counter will stop when it matches the previously setting MR3 match value 36000, the program will lock in a red box below.

```
164  void MN_CtDemoCase2(void)
165  {
166    CT16B2_Init();
167    SN_CT16B2->MR2 = 12000; //Set MR2 value for 1ms period ==> count value = 1000*12 = 12000
168    SN_CT16B2->MR3 = 36000; //Set MR3 value for 3ms period ==> count value = 3000*12 = 36000
169    SN_CT16B2->MCTRL = (mskCT16_MR2IE_EN|mskCT16_MR3IE_EN|mskCT16_MR3STOP_EN);//Set MR2 match interrupt, MR3 match interrupt and TC stop
170    SN_CT16B2->TMRCTRL = (mskCT16_CRST|mskCT16_CM_EDGE_UP);//Set CT16B2 as the up-counting mode.
171    while (SN_CT16B2->TMRCTRL & mskCT16_CRST);  //Wait until timer reset done.
172    SN_CT16B2->TMRCTRL |= mskCT16_CEN_EN;   //Let TC start counting.
173    CT16B2_NvicEnable();//Enable CT16B2's NVIC interrupt.
174    //Use P1.0 to indicate CT16B2 MR2 match period,and P1.1 to indicate CT16B2 MR3 match period.
175    SN_GPIO1->MODE = 0x03;  //Set P1.0/P1.1 as output.
176    SN_GPIO1->DATA = 0x03;  //Default set P1.0/P1.1 as output high.
177    while (1)
178    {
179      if (iwCT16B2_IrqEvent == mskCT16_MR2IF) //Check if MR2 match interrupt occurs
180      {
181        iwCT16B2_IrqEvent = 0;    //Clear MR2 match interrupt variable.
182        SN_GPIO1->BCLR = 0x01;    //Set P1.0 as output low.
183      }
184      if (iwCT16B2_IrqEvent == mskCT16_MR3IF) //Check if MR3 match interrupt occurs
185      {
186        iwCT16B2_IrqEvent = 0;    //Clear MR2 match interrupt variable.
187        SN_GPIO1->BCLR = 0x02;    //Set P1.1 as output low.
188        break;
189      }
190    }
191    while (SN_CT16B2->TC == 36000);// Pass: TC should be always equal to 36000.
192    while (1);                     // Fail: Can't run here.
193  }
```

4.7 Demo Case 3

This example demonstrates CT32B0 PWM function. The TC count will match MR3 every 1ms, and TC will recount to be the PWM cycle time, and make CT32B0_PWM0/PWM1/PWM2 output about 10%/30%/75% duty respectively. Users can observe P1.0 toggle to know when MR3 matches the TC count in time, and the result is shown in Section 5.4.

Demo case must first call CT32B0_Init function to enable PCLK, and set MR3 match value to 12000, to make MR3 match TC as the PWM cycle period 1ms. Set CT32B0_PWM0/PWM1/PWM2 duty by setting MR0/MR1/MR2 match value to 10800/8400/3000, and enable CT32B0_PWM0/PWM1/PWM2 function. When the setting is done, reset the Timer. After the reset completes, then P1.0 is set to low, FW enables TC counting and CT32B0 NVIC interrupt.

In while (1) loop, FW will check whether the MR3 match interrupt is triggered or not, and P1.0 will toggle if yes.

```
218  void MN_CtDemoCase3(void)
219  {
220    CT32B0_Init();
221
222    SN_CT32B0->MR3 = 12000; //Set MR3 value for 1ms PWM period ==> count value = 1000*12 = 12000
223    SN_CT32B0->MR0 = 10800; //Set MR0 value for 10% duty ==> count value = 12000 - (10%*12000) = 10800
224    SN_CT32B0->MR1 = 8400;  //Set MR1 value for 30% duty ==> count value = 12000 - (30%*12000) = 8400
225    SN_CT32B0->MR2 = 3000;  //Set MR2 value for 75% duty ==> count value = 12000 - (75%*12000) = 3000
226    //Enable PWM function, IOs and select the PWM modes
227    SN_CT32B0->PWMCTRL =
228      (mskCT32_PWM0EN_EN|mskCT32_PWM0MODE_1|mskCT32_PWM0IOEN_EN)| //Enable PWM0 function, IO and select as PWM mode 1
229      (mskCT32_PWM1EN_EN|mskCT32_PWM1MODE_1|mskCT32_PWM1IOEN_EN)| //Enable PWM1 function, IO and select as PWM mode 1
230      (mskCT32_PWM2EN_EN|mskCT32_PWM2MODE_1|mskCT32_PWM2IOEN_EN); //Enable PWM2 function, IO and select as PWM mode 1
231    SN_CT32B0->MCTRL = (mskCT32_MR3IE_EN|mskCT32_MR3RST_EN);  //Set MR3 match interrupt and TC rest
232    SN_CT32B0->TMRCTRL = (mskCT32_CRST|mskCT32_CM_EDGE_UP); //Set CT32B0 as the up-counting mode.
233    while (SN_CT32B0->TMRCTRL & mskCT32_CRST);  //Wait until timer reset done.
234    SN_CT32B0->TMRCTRL |= mskCT32_CEN_EN; //Let TC start counting.
235    CT32B0_NvicEnable(); //Enable CT32B0's NVIC interrupt.
236
237    //Use P1.0 to indicate CT32B0 MR3 match interrupt.
238    SN_GPIO1->MODE = 0x01;  //Set P1.0 as output.
239    SN_GPIO1->DATA = 0x00;  //Default set P1.0 as output low.
240    while (1)
241    {
242      if (iwCT32B0_IrqEvent == mskCT32_MR3IF) //Check if MR3 match interrupt occurs
243      {
244        iwCT32B0_IrqEvent = 0;    //Clear MR3 match interrupt variable.
245        SN_GPIO1->DATA ^= 0x01;   //Toggle P1.0
246      }
247    }
248  }
```

4.8 Demo Case 4

This example demonstrates how to use CT32B1 PWM function with External Match every 1ms, and TC will recount to be the PWM cycle time, while CT32B1_PWM0/PWM1/PWM2 output about 0%/30%/70% duty respectively. Users can observe P1.0 toggle to know when MR3 matches the TC count in time, and the results is shown in Section 5.5.

Demo case must first call CT32B1_Init function to enable PCLK, and set MR3 match value to 12000 to make MR3 match TC as the PWM cycle period 1ms. Set CT32B1_PWM0/PWM1/PWM2 duty by setting MR0/MR1/MR2 match value to 0/3600/8400, enable CT32B1_PWM0/PWM1/PWM2 function and External Match Toggle. When the setting is done, reset the Timer. After the reset completes, then P1.0 is set to low, FW enables TC count and CT32B1 NVIC interrupt.

In while (1) loop, FW will check whether the MR3 match interrupt is triggered or not, and P1.0 will toggle if yes.

```
void MN_CtDemoCase4(void)
{
  CT32B1_Init();

  SN_CT32B1->MR3 = 12000; //Set MR3 value for 1ms PWM period ==> count value = 1000*12 = 12000
  SN_CT32B1->MR0 = 0; //Set MR0 value for 0% phase delay ==> count value = 0
  SN_CT32B1->MR1 = 3600;  //Set MR1 value for 30% phase delay ==> count value = 30%*12000 = 3600
  SN_CT32B1->MR2 = 8400;  //Set MR2 value for 70% phase delay ==> count value = 70%*12000 = 8400
  SN_CT32B1->EM = //Set External Match for EM0, EM1, EM2 as toggle
    (mskCT32_EMC0_TOGGLE)|  //Enable PWM0 External Match to toggle PWM0.
    (mskCT32_EMC1_TOGGLE)|  //Enable PWM1 External Match to toggle PWM1.
    (mskCT32_EMC2_TOGGLE);  //Enable PWM2 External Match to toggle PWM2.
  SN_CT32B1->PWMCTRL = //Enable PWM External Match and IOs.
    (mskCT32_PWM0EN_EM0|mskCT32_PWM0IOEN_EN)| //Enable PWM0 External Match and IO.
    (mskCT32_PWM1EN_EM1|mskCT32_PWM1IOEN_EN)| //Enable PWM1 External Match and IO.
    (mskCT32_PWM2EN_EM2|mskCT32_PWM2IOEN_EN); //Enable PWM2 External Match and IO.
  SN_CT32B1->MCTRL = (mskCT32_MR3IE_EN|mskCT32_MR3RST_EN);//Set MR3 match interrupt and TC rest
  SN_CT32B1->TMRCTRL = (mskCT32_CRST|mskCT32_CM_EDGE_UP);//Set CT32B1 as the up-counting mode.
  while (SN_CT32B1->TMRCTRL & mskCT32_CRST);  //Wait until timer reset done.
  SN_CT32B1->TMRCTRL |= mskCT32_CEN_EN; //Let TC start counting.
  CT32B1_NvicEnable();//Enable CT32B1's NVIC interrupt.
  SN_GPIO1->MODE = 0x01;  //Set P1.0 as output. //Use P1.0 to indicate CT32B1 MR3 match interrupt.
  SN_GPIO1->DATA = 0x00;  //Default set P1.0 as output low.
  while (1)
  {
    if (iwCT32B1_IrqEvent == mskCT32_MR3IF) //Check if MR3 match interrupt occurs
    {
      iwCT32B1_IrqEvent = 0;    //Clear MR3 match interrupt variable.
      SN_GPIO1->DATA ^= 0x01;   //Toggle P1.0
    }
  }
}
```

4.9 Demo Case 5

This example demonstrates CT32B2 Counter input counting function. Users should connect P1.0 to CT32B2_CAP0. The P1.0 is programmed to output the waveform which is the input source of CT32B2_CAP0, the results is shown in Section 5.6.

Demo case must first call CT32B2_Init function to enable PCLK, and set MR0 match value to 100, which is also the value to end CAP0 Counting. When the setting is done, reset the Timer. After the reset completes, FW enables TC count and CT32B2 NVIC interrupt.

FW makes P1.0 to toggle 100 times to generate 100 rising edges immediately, and then check whether the MR3 match interrupt is triggered or not. If yes, FW will then check whether the TC value is exactly equal to 100, and the program will lock in a red box below if yes.

```
347  void MN_CtDemoCase5(void)
348  {
349    uint32_t wToggleCnt = 0;    //Indicates the I/O toggle count.
350
351    //Let P1.0 be the CAP0 counter's input source.
352    SN_GPIO1->MODE = 0x01;  //Set P1.0 as output mode.
353    SN_GPIO1->DATA = 0x00;  //Set P1.0 as output low.
354
355    CT32B2_Init();
356    SN_CT32B2->MR0 = 100;//Set MR0 match value as 100.
357    SN_CT32B2->CNTCTRL = mskCT32_CTM_CNTER_RISING|mskCT32_CIS;//Set Counter Control as rising edge counter mode.
358    SN_CT32B2->CAPCTRL = mskCT32_CAP0EN_EN; //Enable CAP0 function.
359    SN_CT32B2->MCTRL = mskCT32_MR0IE_EN;//Enable MR0 match interrupt.
360    SN_CT32B2->TMRCTRL = mskCT32_CRST;//Set CT32B2 to reset TC value.
361    while (SN_CT32B2->TMRCTRL & mskCT32_CRST);  //Wait until timer reset done.
362    SN_CT32B2->TMRCTRL |= mskCT32_CEN_EN; //Let TC start CAP0 Counter function.
363    CT32B2_NvicEnable();//Enable CT32B2's NVIC interrupt.
364    //Let P1.0 toggle 100 times to generate 100 times of rising edge.
365    for (wToggleCnt = 0; wToggleCnt < 100; wToggleCnt++)
366    {
367      SN_GPIO1->BSET = 0x01;      //Set P1.0 as output high.
368      UT_DelayNx10us(1);          //Delay 10us
369      SN_GPIO1->BCLR = 0x01;      //Set P1.0 as output low.
370      UT_DelayNx10us(1);          //Delay 10us
371    }
372    if (iwCT32B2_IrqEvent == mskCT32_MR0IF)// Check whether TC matches MR0
373    {
374      iwCT32B2_IrqEvent = 0;  //Clear MR0 match interrupt variable.
375      //Check whether wToggleCnt is equal to TC value.
376      if (SN_CT32B2->TC == wToggleCnt)
377        while (1);                //Pass point
378    }
379    while (1);                    //Fail point
380  }
```

4.10 Demo Case 6

This example demonstrates CT16B0 input Capture function. Users should connect CT32B0_PWM0 to CT16B0_CAP0, the result is shown in Section 5.7.

Firstly, enable CT32B0 PWM0 function, set MR0 match value to (12000-1), and enable External Match Toggle to make PWM0 output the 1KHz square wave.

Secondly, enable CT16B0 CAP0 capture function, set counter mode and enable Rising Edge Capture/Falling Edge Capture/CAP0 Interrupt/CAP0 input function, then reset the Timer. After the reset completes, FW enables TC count and waits for CAP0 capture.

Thirdly, start CT32B0 PWM0 output.

Fourthly, check CAP0 interrupt trigger signal.

Since the CAP0 internal count is not synchronized with the PWM0 output, so ignores the first retrieved CAP0 value, and then begins to retrieve CAP0 value stored in the buffer. This example retrieves a total of 20 results. After the completion of 20 times, start checking whether the existence of each data buffer is equal to 12000 (MR0 value + 1) or not, the program will lock in a red box below if yes.

```
395  void MN_CtDemoCase6(void)
396  {
397    uint32_t wCnt = 0;         //Indicates capture count times.
398    uint16_t wPreCap0 = 0;     //Indicates the previous  CAP0 value.
399    uint16_t wCap0Buffer[20]; //Used to save delta CAP0 values of 20 times.
400
401    /***** Step 1: Setup the 1KHz toggle in CT32B0. *****/
402    CT32B0_Init();
403    //Set MR0 match value for 1KHz toggle rate output by External Match toggle method.
404    //Thus, the match value should be (12000 - 1)@HCLK = 12MHz.
405    SN_CT32B0->MR0 = 12000 - 1;
406    //Set External Match for EM0 toggle.
407    SN_CT32B0->EM = mskCT32_EMC0_TOGGLE;
408    //Enable PWM0 External Match and IO.
409    SN_CT32B0->PWMCTRL = (mskCT32_PWM0EN_EM0|mskCT32_PWM0IOEN_EN);
410    //Set MR0 match to reset TC.
411    SN_CT32B0->MCTRL = mskCT32_MR0RST_EN;
412    //Set CT32B0 as the up-counting mode.
413    SN_CT32B0->TMRCTRL = (mskCT32_CRST|mskCT32_CM_EDGE_UP);
414
415    /***** Step 2: Setup CAP0 timer capture in CT16B0.*****/
416    CT16B0_Init();
417    SN_CT16B0->CNTCTRL = (mskCT16_CTM_TIMER|mskCT16_CIS); //Set Counter Control as timer counter mode.
418    SN_CT16B0->CAPCTRL = //Enable CAP0 function.
419        (mskCT16_CAP0RE_EN)|  //Enable rising edge capture.
420        (mskCT16_CAP0FE_EN)|  //Enable falling dge capture.
421        (mskCT16_CAP0IE_EN)|  //Enable CAP0 interrupt.
422        (mskCT16_CAP0EN_EN);  //Enable CAP0 input function.
423    SN_CT16B0->TMRCTRL = mskCT16_CRST;  //Set CT16B0 to reset TC value.
424    while (SN_CT16B0->TMRCTRL & mskCT16_CRST);  //Wait until timer reset done.
425    SN_CT16B0->TMRCTRL |= mskCT16_CEN_EN;    //Let TC start CAP0 Counter function.
426    //Enable CT16B0's NVIC interrupt.
427    //CT16B0_NvicEnable();
```

```
429   /***** Step 3: Start to toggle 1KHz from CT32B0 PWM0. *****/
430   while (SN_CT32B0->TMRCTRL & mskCT32_CRST);  //Wait until timer reset done.
431   SN_CT32B0->TMRCTRL |= mskCT32_CEN_EN;      //Let TC start PWM function.
432
433   /***** Step 4: Let CT16B0 CAP0 capture for 20 times *****/
434   while (1) //Ignore the first time of CAP0 capture but save it first.
435   {
436     if (SN_CT16B0->RIS & mskCT16_CAP0IF)//Check whether CAP0 capture has done
437     {
438         SN_CT16B0->IC = mskCT16_CAP0IC; //Clear CAP0 status.
439         wPreCap0 = SN_CT16B0->CAP0;     //Save the CAP0 value.
440         break;
441     }
442   }
443   while (1) //Keep retrieving the delta value of each CAP0 status occurs for 20 times.
444   {
445     if (SN_CT16B0->RIS & mskCT16_CAP0IF)//Check whether CAP0 capture has done
446     {
447         wCap0Buffer[wCnt] = SN_CT16B0->CAP0 - wPreCap0; //Save the delta value into wCap0Buffer
448         wPreCap0 = SN_CT16B0->CAP0;     //Save the CAP0 value.
449         SN_CT16B0->IC = mskCT16_CAP0IC; //Clear CAP0 status.
450         wCnt++;                         //Increase loop count.
451         if (wCnt == 20)                 //Check if already retrieving 20 times
452           break;
453     }
454   }
455   for (wCnt = 0; wCnt < 20; wCnt++)   //Check whether all CAP0 delta in wCap0Buffer matches 12000.
456   {
457     if (wCap0Buffer[wCnt] != 12000)//Check if matches 12000.
458       while (1);                //Fail point
459   }
460   while (1);                    //Pass point
461 }
```

4.11 Demo Case 7

This example demonstrates CT16B0 PWM function. The TC count will match MR9 every 1ms, and TC will recount to be the PWM cycle time, and make CT16B0_PWM0/PWM1/PWM2 output about 10%/30%/75% duty respectively. Users can observe P1.0 toggle to know when MR3 matches the TC count in time, and the result is shown in Section 5.8.

Demo case must first call CT16B0_Init function to enable PCLK, and set MR9 match value to 12000, to make MR9 match TC as the PWM cycle period 1ms. Set PWM0/PWM1/PWM2 duty by setting MR0/MR1/MR2 match value to 10800/8400/3000, and enable PWM0/PWM1/PWM2 function. When the setting is done, reset the Timer. After the reset completes, then P1.0 is set to low, FW enables TC counting and CT16B0 NVIC interrupt.

In while (1) loop, FW will check whether the MR9 match interrupt is triggered or not, and the P1.0 will toggle if yes.

```
382  void MN_CtDemoCase7(void)
383  {
384    SN_SYS0->SWDCTRL = 1;   //Disable SWD pin
385
386    CT16B0_Init();
387
388    //Set MR9 value for 1ms PWM period ==> count value = 1000*12 = 12000
389    SN_CT16B0->MR9 = 12000;
390
391    //Set MR0 value for 10% duty ==> count value = 12000 - (10%*12000) = 10800
392    SN_CT16B0->MR0 = 10800;
393
394    //Set MR1 value for 30% duty ==> count value = 12000 - (30%*12000) = 8400
395    SN_CT16B0->MR1 = 8400;
396
397    //Set MR2 value for 75% duty ==> count value = 12000 - (75%*12000) = 3000
398    SN_CT16B0->MR2 = 3000;
399
400    //Enable PWM function, IOs and select the PWM modes
401    SN_CT16B0->PWMCTRL =
402      (mskCT16_PWM0EN_EN|mskCT16_PWM0MODE_1|mskCT16_PWM0IOEN_EN)| //Enable PWM0 function, IO and select as PWM mode 1
403      (mskCT16_PWM1EN_EN|mskCT16_PWM1MODE_1|mskCT16_PWM1IOEN_EN)| //Enable PWM1 function, IO and select as PWM mode 1
404      (mskCT16_PWM2EN_EN|mskCT16_PWM2MODE_1|mskCT16_PWM2IOEN_EN); //Enable PWM2 function, IO and select as PWM mode 1
405
406    //Set MR9 match interrupt and TC rest
407    SN_CT16B0->MCTRL = (mskCT16_MR9IE_EN|mskCT16_MR9RST_EN);
408
409    //Set CT16B0 as the up-counting mode.
410    SN_CT16B0->TMRCTRL = (mskCT16_CRST|mskCT16_CM_EDGE_UP);
411
412    //Wait until timer reset done.
413    while (SN_CT16B0->TMRCTRL & mskCT16_CRST);
414
415    //Let TC start counting.
416    SN_CT16B0->TMRCTRL |= mskCT16_CEN_EN;
417
418    //Enable CT16B0's NVIC interrupt.
419    CT16B0_NvicEnable();
420
421    //Use P1.0 to indicate CT32B0 MR3 match interrupt.
422    SN_GPIO1->MODE = 0x01;  //Set P1.0 as output.
423    SN_GPIO1->DATA = 0x00;  //Default set P1.0 as output low.
424    while (1)
425    {
426      if (iwCT16B0_IrqEvent == mskCT16_MR9IF) //Check if MR9 match interrupt occurs
427      {
428        iwCT16B0_IrqEvent = 0;    //Clear MR9 match interrupt variable.
429        SN_GPIO1->DATA ^= 0x01;   //Toggle P1.0
430      }
431    }
432  }
```

4.12 Demo Case 8

This example demonstrates how to use CT16B0 PWM function with External Match. The TC matches MR9 every 1ms, and TC will recount to be the PWM cycle time, while CT16B0_PWM0/ PWM1/PWM2 output about 0%/30%/70% duty respectively. Users can observe P1.0 toggle to know when MR9 matches the TC count in time, and the results is shown in Section 5.9.

Demo case must first call CT16B0_Init function to enable PCLK, and set MR9 match value to 12000 to make MR9 match TC as the PWM cycle period 1ms. Set CT16B0_PWM0/PWM1/PWM2 duty by setting MR0/MR1/MR2 match value to 0/3600/8400, enable CT16B0_PWM0/PWM1/PWM2 function and External Match Toggle. When the setting is done, reset the Timer. After the reset completes, then P1.0 is set to low, FW enables TC count and CT16B0 NVIC interrupt.

In while (1) loop, FW will check whether the MR9 match interrupt is triggered or not, and P1.0 will toggle if yes.

```
457  void MN_CtDemoCase8(void)
458  {
459    SN_SYS0->SWDCTRL = 1;   //Disable SWD pin
460
461    CT16B0_Init();
462
463    //Set MR9 value for 1ms PWM period ==> count value = 1000*12 = 12000
464    SN_CT16B0->MR9 = 12000;
465    //Set MR0 value for 0% phase delay ==> count value = 0
466    SN_CT16B0->MR0 = 0;
467    //Set MR1 value for 30% phase delay ==> count value = 30%*12000 = 3600
468    SN_CT16B0->MR1 = 3600;
469    //Set MR2 value for 70% phase delay ==> count value = 70%*12000 = 8400
470    SN_CT16B0->MR2 = 8400;
471
472    //Set External Match for EM0, EM1, EM2 as toggle
473    SN_CT16B0->EM =
474      (mskCT16_EMC0_TOGGLE)|  //Enable PWM0 External Match to toggle PWM0.
475      (mskCT16_EMC1_TOGGLE)|  //Enable PWM1 External Match to toggle PWM1.
476      (mskCT16_EMC2_TOGGLE);  //Enable PWM2 External Match to toggle PWM2.
477
478    //Enable PWM External Match and IOs.
479    SN_CT16B0->PWMCTRL =
480      (mskCT16_PWM0EN_EM0|mskCT16_PWM0IOEN_EN)| //Enable PWM0 External Match and IO.
481      (mskCT16_PWM1EN_EM1|mskCT16_PWM1IOEN_EN)| //Enable PWM1 External Match and IO.
482      (mskCT16_PWM2EN_EM2|mskCT16_PWM2IOEN_EN); //Enable PWM2 External Match and IO.
483
484    //Set MR3 match interrupt and TC rest
485    SN_CT16B0->MCTRL = (mskCT16_MR9IE_EN|mskCT16_MR9RST_EN);
486    //Set CT16B0 as the up-counting mode.
487    SN_CT16B0->TMRCTRL = (mskCT16_CRST|mskCT16_CM_EDGE_UP);
488    //Wait until timer reset done.
489    while (SN_CT16B1->TMRCTRL & mskCT16_CRST);
490
491    //Let TC start counting.
492    SN_CT16B0->TMRCTRL |= mskCT16_CEN_EN;
493    //Enable CT32B1's NVIC interrupt.
494    CT16B0_NvicEnable();
495
496    //Use P1.0 to indicate CT16B0 MR9 match interrupt.
497    SN_GPIO1->MODE = 0x01;  //Set P1.0 as output.
498    SN_GPIO1->DATA = 0x00;  //Default set P1.0 as output low.
499    while (1)
500    {
501      if (iwCT16B0_IrqEvent == mskCT16_MR9IF) //Check if MR9 match interrupt occurs
502      {
503        iwCT16B0_IrqEvent = 0;    //Clear MR9 match interrupt variable.
504        SN_GPIO1->DATA ^= 0x01;   //Toggle P1.0
505      }
506    }
507  }
```

4.13 Demo Case 9

This example demonstrates CT16B1 Counter input counting function. Users should connect P1.0 to CT16B1_CAP0. The P1.0 is programmed to output the waveform which is the input source of CT16B1_CAP0, the results is shown in Section 5.10.

Demo case must first call CT16B1_Init function to enable PCLK, and set MR0 match value to 100, which is also the value to end CAP0 Counting. When the setting is done, reset the Timer. After the reset completes, FW enables TC count and CT32B2 NVIC interrupt.

FW makes P1.0 to toggle 100 times to generate 100 clocks immediately, and then check whether the MR0 match interrupt is triggered or not. If yes, FW will then check whether the TC value is exactly equal to 100, and the program will lock in a red box below if yes.

```
402  void MN_CtDemoCase9(void)
403  {
404    uint32_t wToggleCnt = 0;    //Indicates the I/O toggle count.
405
406    //Let P1.0 be the CAP0 counter's input source.
407    SN_GPIO1->MODE = 0x01;  //Set P1.0 as output mode.
408    SN_GPIO1->DATA = 0x00;  //Set P1.0 as output low.
409
410    CT16B1_Init();
411
412    //Set MR0 match value as 100.
413    SN_CT16B1->MR0 = 100;
414
415    //Set Counter Control as rising edge counter mode.
416    SN_CT16B1->CNTCTRL = mskCT16_CTM_CNTER_RISING|mskCT16_CIS;
417
418    //Enable CAP0 function.
419    SN_CT16B1->CAPCTRL = mskCT16_CAP0EN_EN;
420
421    //Enable MR0 match interrupt.
422    SN_CT16B1->MCTRL = mskCT16_MR0IE_EN;
423
424    //Set CT16B1 to reset TC value.
425    SN_CT16B1->TMRCTRL = mskCT16_CRST;
426
427    //Wait until timer reset done.
428    while (SN_CT16B1->TMRCTRL & mskCT16_CRST);
429
430    //Let TC start CAP0 Counter function.
431    SN_CT16B1->TMRCTRL |= mskCT16_CEN_EN;
432
433    //Enable CT16B1's NVIC interrupt.
434    CT16B1_NvicEnable();
435
436    //Let P1.0 toggle 100 times to generate 100 times of rising edge.
437    for (wToggleCnt = 0; wToggleCnt < 100; wToggleCnt++)
438    {
439      SN_GPIO1->BSET = 0x01;      //Set P1.0 as output high.
440      UT_DelayNx10us(1);          //Delay 10us
441      SN_GPIO1->BCLR = 0x01;      //Set P1.0 as output low.
442      UT_DelayNx10us(1);          //Delay 10us
443    }
444
445    // Check whether TC matches MR0
446    if (iwCT16B1_IrqEvent == mskCT16_MR0IF)
447    {
448      iwCT16B1_IrqEvent = 0;  //Clear MR0 match interrupt variable.
449      //Check whether wToggleCnt is equal to TC value.
450      if (SN_CT16B1->TC == wToggleCnt)
451        while (1);                //Pass point
452    }
453    while (1);                    //Fail point
454  }
```

4.14 Demo Case 10

This example demonstrates CT16B0 input Capture function. Users should connect CT16B1_PWM0 to CT16B0_CAP0, the result is shown in Section 5.11.

Firstly, enable CT16B1 PWM0 function, set MR0 match value to (12000-1), and enable External Match Toggle to make PWM0 output the 1KHz square wave.

Secondly, enable CT16B0 CAP0 capture function, set counter mode and enable Rising Edge Capture/Falling Edge Capture/CAP0 Interrupt/CAP0 input function, and then reset the Timer. After the reset completes, FW enables TC count and waits for CAP0 capture.

Thirdly, start CT16B1 PWM0 output.

Fourthly, check CAP0 interrupt trigger signal.

Since the CAP0 internal count is not synchronized with the PWM0 output, so ignores the first retrieved CAP0 value, and then begins to retrieve CAP0 value stored in the buffer. This example retrieves 20 results totally. After the completion of 20 times, start checking whether each data buffer is equal to 12000 (MR0 value + 1) or not, the program will lock in a red box below if yes.

```
468  void MN_CtDemoCase10(void)
469  {
470    uint32_t wCnt = 0;         //Indicates capture count times.
471    uint16_t wPreCap0 = 0;     //Indicates the previous  CAP0 value.
472    uint16_t wCap0Buffer[20]; //Used to save delta CAP0 values of 20 times.
473
474    /***** Step 1: Setup the 1KHz toggle in CT16B1. *****/
475    CT16B1_Init();
476
477    //Set MR0 match value for 1KHz toggle rate output by External Match toggle method.
478    //Thus, the match value should be (12000 - 1)@HCLK = 12MHz.
479    SN_CT16B1->MR0 = 12000 - 1;
480    //Set External Match for EM0 toggle.
481    SN_CT16B1->EM = mskCT16_EMC0_TOGGLE;
482    //Enable PWM0 External Match and IO.
483    SN_CT16B1->PWMCTRL = (mskCT16_PWM0EN_EM0|mskCT16_PWM0IOEN_EN);
484    //Set MR0 match to reset TC.
485    SN_CT16B1->MCTRL = mskCT16_MR0RST_EN;
486    //Set CT32B0 as the up-counting mode.
487    SN_CT16B1->TMRCTRL = (mskCT16_CRST|mskCT16_CM_EDGE_UP);
488
489    /***** Step 2: Setup CAP0 timer capture in CT16B0.*****/
490    CT16B0_Init();
491
492    //Set Counter Control as timer counter mode.
493    SN_CT16B0->CNTCTRL = (mskCT16_CTM_TIMER|mskCT16_CIS);
494
495    //Enable CAP0 function.
496    SN_CT16B0->CAPCTRL =
497        (mskCT16_CAP0RE_EN)|  //Enable rising edge capture.
498        (mskCT16_CAP0FE_EN)|  //Enable falling dge capture.
499        (mskCT16_CAP0IE_EN)|  //Enable CAP0 interrupt.
500        (mskCT16_CAP0EN_EN);  //Enable CAP0 input function.
501
502    //Set CT16B0 to reset TC value.
503    SN_CT16B0->TMRCTRL = mskCT16_CRST;
504
505    //Wait until timer reset done.
506    while (SN_CT16B0->TMRCTRL & mskCT16_CRST);
507
508    //Let TC start CAP0 Counter function.
509    SN_CT16B0->TMRCTRL |= mskCT16_CEN_EN;
510
511    //Enable CT16B0's NVIC interrupt.
512    //CT16B0_NvicEnable();
513
514    /***** Step 3: Start to toggle 1KHz from CT16B1 PWM0. *****/
515    //Wait until timer reset done.
516    while (SN_CT16B1->TMRCTRL & mskCT16_CRST);
517
518    //Let TC start PWM function.
519    SN_CT16B1->TMRCTRL |= mskCT16_CEN_EN;
```

```
520
521
522   /***** Step 4: Let CT16B0 CAP0 capture for 20 times *****/
523   //Ignore the first time of CAP0 capture but save it first.
524   while (1)
525   {
526     if (SN_CT16B0->RIS & mskCT16_CAP0IF)//Check whether CAP0 capture has done
527     {
528         SN_CT16B0->IC = mskCT16_CAP0IC; //Clear CAP0 status.
529         wPreCap0 = SN_CT16B0->CAP0;     //Save the CAP0 value.
530         break;
531     }
532   }
533
534   //Keep retrieving the delta value of each CAP0 status occurs for 20 times.
535   while (1)
536   {
537     if (SN_CT16B0->RIS & mskCT16_CAP0IF)//Check whether CAP0 capture has done
538     {
539         wCap0Buffer[wCnt] = SN_CT16B0->CAP0 - wPreCap0; //Save the delta value into wCap0Buffer
540         wPreCap0 = SN_CT16B0->CAP0;     //Save the CAP0 value.
541         SN_CT16B0->IC = mskCT16_CAP0IC; //Clear CAP0 status.
542         wCnt++;                         //Increase loop count.
543         if (wCnt == 20)                 //Check if already retrieving 20 times
544           break;
545     }
546   }
547   //Check whether all CAP0 delta in wCap0Buffer matches 12000.
548   for (wCnt = 0; wCnt < 20; wCnt++)
549   {
550     if (wCap0Buffer[wCnt] != 12000)//Check if matches 12000.
551       while (1);                //Fail point
552   }
553   while (1);                    //Pass point
554 }
```

4.15 Demo Case 11

This example demonstrates CT16B2 PWM Dead-band function. The TC count will match MR9 every 2ms, and TC will recount to be the PWM cycle time. Users can observe P0.0 toggle to know when MR9 matches the TC count in time, the result is shown in Section 5.12.

Demo case must first call CT16B2_Init function to enable PCLK, and set MR9 match value to 12000 to make MR9 match TC as the PWM cycle period 2ms. Set CT16B2_PWM0N/PWM1N/PWM2N/PWM3N Dead-band period by setting PWM0NDB/PWM1NDB/PWM2NDB/PWM3NDB value to 0/1000/1000/1000, and enable CT16B0_PWM0N/PWM1N/PWM2N/PWM3N function. When the setting is done, reset the Timer. After the reset completes, then P0.0 is set to low, FW enables TC count and CT16B2 NVIC interrupt.

In while (1) loop, FW will check whether the MR9 match interrupt is triggered or not, and P0.0 will toggle if yes.

```
582  void MN_CtDemoCase11(void)
583  {
584    CT16B2_Init();
585
586    //Set MR9 value for 2ms PWM period ==> count value = 1000*12 = 12000
587    SN_CT16B2->MR9 = 12000;
588
589    SN_CT16B2->MR0 = 8000;          //Set MR0 match value as 8000.
590    SN_CT16B2->MR1 = 8000;          //Set MR1 match value as 8000.
591    SN_CT16B2->MR2 = 8000;          //Set MR2 match value as 8000.
592    SN_CT16B2->MR3 = 8000;          //Set MR3 match value as 8000.
593
594    //Set MR9 match interrupt
595    SN_CT16B2->MCTRL = mskCT16_MR9IE_EN;
596
597    //Enable PWM function, IOs and select the PWM modes
598    SN_CT16B2->PWMCTRL =
599      (mskCT16_PWM0EN_EN|mskCT16_PWM0MODE_1|mskCT16_PWM0IOEN_EN|mskCT16_PWM0N_MODE3)| //Enable PWM0/PWM0N function, IO and select as PWM mode 1
600      (mskCT16_PWM1EN_EN|mskCT16_PWM1MODE_1|mskCT16_PWM1IOEN_EN|mskCT16_PWM1N_MODE1)| //Enable PWM1/PWM1N function, IO and select as PWM mode 1
601      (mskCT16_PWM2EN_EN|mskCT16_PWM2MODE_1|mskCT16_PWM2IOEN_EN|mskCT16_PWM2N_MODE2)| //Enable PWM2/PWM2N function, IO and select as PWM mode 1
602      (mskCT16_PWM3EN_EN|mskCT16_PWM3MODE_1|mskCT16_PWM3IOEN_EN|mskCT16_PWM3N_MODE3); //Enable PWM3/PWM3N function, IO and select as PWM mode 1
603
604    SN_CT16B2->PWM0NDB = 0;           //Set PWM0N Dead-band value as 0.
605    SN_CT16B2->PWM1NDB = 1000;        //Set PWM1N Dead-band value as 1000.
606    SN_CT16B2->PWM2NDB = 1000;        //Set PWM2N Dead-band value as 1000.
607    SN_CT16B2->PWM3NDB = 1000;        //Set PWM3N Dead-band value as 1000.
608
609    //Set CT16B2 as the Center-aligned mode1.
610    SN_CT16B2->TMRCTRL = (mskCT16_CRST|mskCT16_CM_CENTER_DOWN);
611    //Wait until timer reset done.
612    while (SN_CT16B2->TMRCTRL & mskCT16_CRST);
613    //Let TC start counting.
614    SN_CT16B2->TMRCTRL |= mskCT16_CEN_EN;
615
616    //Enable CT16B0's NVIC interrupt.
617    CT16B2_NvicEnable();
618
619    //Use P0.0 to indicate CT16B2 MR9 match interrupt.
620    SN_GPIO0->MODE = 0x01;  //Set P0.0 as output.
621    SN_GPIO0->DATA = 0x00;  //Default set P0.0 as output low.
622    while (1)
623    {
624      if (iwCT16B2_IrqEvent == mskCT16_MR9IF) //Check if MR9 match interrupt occurs
625      {
626        iwCT16B2_IrqEvent = 0;    //Clear MR9 match interrupt variable.
627        SN_GPIO0->DATA ^= 0x01;   //Toggle P0.0
628      }
629    }
630  }
```

4.16 Demo Case 12

This example demonstrates CT16B1 PWM function. The TC count will match MR23 every 1ms, and TC will recount to be the PWM cycle time, and make CT16B1_PWM0/PWM1/PWM2 output about 10%/30%/75% duty respectively. Users can observe P1.0 toggle to know when MR23 matches the TC count in time, and the result is shown in Section 5.13.

Demo case must first call CT16B1_Init function to enable PCLK, and set MR23 match value to 12000, to make MR23 match TC as the PWM cycle period 1ms. Set CT16B1_PWM0/PWM1/PWM2 duty by setting MR0/MR1/MR2 match value to 10800/8400/3000, and enable CT16B1_PWM0/PWM1/PWM2 function. When the setting is done, reset the Timer. After the reset completes, then P1.0 is set to low, FW enables TC counting and CT16B1 NVIC interrupt.

In while (1) loop, FW will check whether the MR23 match interrupt is triggered or not, and the P1.0 will toggle if yes.

```
void MN_CtDemoCase12(void)
{
  //SN_SYS0->SWDCTRL = 1;   //Disable SWD pin

  CT16B1_Init();

  //Set MR23 value for 1ms PWM period ==> count value = 1000*12 = 12000
  SN_CT16B1->MR23 = 12000;

  //Set MR0 value for 10% duty ==> count value = 12000 - (10%*12000) = 10800
  SN_CT16B1->MR0 = 10800;

  //Set MR1 value for 30% duty ==> count value = 12000 - (30%*12000) = 8400
  SN_CT16B1->MR1 = 8400;

  //Set MR2 value for 75% duty ==> count value = 12000 - (75%*12000) = 3000
  SN_CT16B1->MR2 = 3000;

  //Enable PWM function, IOs and select the PWM modes
  SN_CT16B1->PWMENB = (mskCT16_PWM0EN_EN|mskCT16_PWM1EN_EN|mskCT16_PWM2EN_EN);//Enable PWM0/1/2 function
  SN_CT16B1->PWMIOENB = (mskCT16_PWM0IOEN_EN|mskCT16_PWM1IOEN_EN|mskCT16_PWM2IOEN_EN);//Enable PWM0/1/2 IO
  SN_CT16B1->PWMCTRL = (mskCT16_PWM0MODE_1|mskCT16_PWM1MODE_1|mskCT16_PWM2MODE_1);  //PWM0/1/2  select as PWM mode 1

  //Set MR23 match interrupt and TC rest
  SN_CT16B1->MCTRL3 = (mskCT16_MR23IE_EN|mskCT16_MR23RST_EN);

  //Set CT16B0 as the up-counting mode.
  SN_CT16B1->TMRCTRL = (mskCT16_CRST|mskCT16_CM_EDGE_UP);

  //Wait until timer reset done.
  while (SN_CT16B1->TMRCTRL & mskCT16_CRST);

  //Let TC start counting.
  SN_CT16B1->TMRCTRL |= mskCT16_CEN_EN;

  //Enable CT16B0's NVIC interrupt.
  CT16B1_NvicEnable();

  //Use P1.0 to indicate CT32B0 MR3 match interrupt.
  SN_GPIO1->MODE = 0x01;  //Set P1.0 as output.
  SN_GPIO1->DATA = 0x00;  //Default set P1.0 as output low.
  while (1)
  {
    if (iwCT16B1_IrqEvent == mskCT16_MR23IF) //Check if MR9 match interrupt occurs
    {
      iwCT16B1_IrqEvent = 0;    //Clear MR9 match interrupt variable.
      SN_GPIO1->DATA ^= 0x01;   //Toggle P1.0
    }
  }
}
```

4.17 Demo Case 13

This example demonstrates how to use CT16B1 PWM function with External Match every 1ms, and TC will recount to be the PWM cycle time, while CT16B1_PWM0/PWM1/PWM2 output about 0%/30%/70% duty respectively. Users can observe P1.0 toggle to know when MR23 matches the TC count in time, and the results is shown in Section 5.14.

Demo case must first call CT16B1_Init function to enable PCLK, and set MR23 match value to 12000 to make MR23 match TC as the PWM cycle period 1ms. Set CT16B1_PWM0/PWM1/PWM2 duty by setting MR0/MR1/MR2 match value to 0/3600/8400, enable CT16B1_PWM0/PWM1/PWM2 function and External Match Toggle. When the setting is done, reset the Timer. After the reset completes, then P1.0 is set to low, FW enables TC count and CT16B1 NVIC interrupt.

In while (1) loop, FW will check whether the MR23 match interrupt is triggered or not, and P1.0 will toggle if yes.

```
void MN_CtDemoCase13(void)
{
  CT16B1_Init();

  //Set MR23 value for 1ms PWM period ==> count value = 1000*12 = 12000
  SN_CT16B1->MR23 = 12000;
  //Set MR0 value for 0% phase delay ==> count value = 0
  SN_CT16B1->MR0 = 0;
  //Set MR1 value for 30% phase delay ==> count value = 30%*12000 = 3600
  SN_CT16B1->MR1 = 3600;
  //Set MR2 value for 70% phase delay ==> count value = 70%*12000 = 8400
  SN_CT16B1->MR2 = 8400;

  //Set External Match for EMC0, EMC1, EMC2 as toggle
  SN_CT16B1->EMC =
    (mskCT16_EMC0_TOGGLE)|  //Enable PWM0 External Match to toggle PWM0.
    (mskCT16_EMC1_TOGGLE)|  //Enable PWM1 External Match to toggle PWM1.
    (mskCT16_EMC2_TOGGLE);  //Enable PWM2 External Match to toggle PWM2.

  //Enable PWM External Match and IOs.
  SN_CT16B1->PWMENB =
    (mskCT16_PWM0EN_EM0|//Enable PWM0 External Match and IO.
    mskCT16_PWM1EN_EM1| //Enable PWM1 External Match and IO.
    mskCT16_PWM2EN_EM2);  //Enable PWM2 External Match and IO.

  SN_CT16B1->PWMIOENB =
    (mskCT16_PWM0IOEN_EN|mskCT16_PWM1IOEN_EN|mskCT16_PWM2IOEN_EN);//Enable PWM0/1/2 IO

  SN_CT16B1->PWMCTRL =
    (mskCT16_PWM0MODE_1|mskCT16_PWM1MODE_1|mskCT16_PWM2MODE_1); //PWM0/1/2 select as PWM mode 1

  //Set MR23 match interrupt and TC rest
  SN_CT16B1->MCTRL3 = (mskCT16_MR23IE_EN|mskCT16_MR23RST_EN);
  //Set CT16B1 as the up-counting mode.
  SN_CT16B1->TMRCTRL = (mskCT16_CRST|mskCT16_CM_EDGE_UP);
  //Wait until timer reset done.
  while (SN_CT16B1->TMRCTRL & mskCT16_CRST);

  //Let TC start counting.
  SN_CT16B1->TMRCTRL |= mskCT16_CEN_EN;
  //Enable CT32B1's NVIC interrupt.
  CT16B1_NvicEnable();

  //Use P1.0 to indicate CT16B0 MR9 match interrupt.
  SN_GPIO1->MODE = 0x01;  //Set P1.0 as output.
  SN_GPIO1->DATA = 0x00;  //Default set P1.0 as output low.
  while (1)
  {
    if (iwCT16B1_IrqEvent == mskCT16_MR23IF) //Check if MR9 match interrupt occurs
    {
      iwCT16B1_IrqEvent = 0;    //Clear MR9 match interrupt variable.
      SN_GPIO1->DATA ^= 0x01;   //Toggle P1.0
    }
  }
```

4.18 Demo Case 14

This example demonstrates CT16B0 Counter input counting function. Users should connect P1.0 to CT16B0_CAP0. The P1.0 is programmed to output the waveform which is the input source of CT16B0_CAP0, the results is shown in Section 5.15.

Demo case must first call CT16B0_Init function to enable PCLK, and set MR0 match value to 100, which is also the value to end CAP0 Counting. When the setting is done, reset the Timer. After the reset completes, FW enables TC count and CT16B0 NVIC interrupt.

FW makes P1.0 to toggle 100 times to generate 100 clocks edges immediately, and then check whether the MR0 match interrupt is triggered or not. If yes, FW will then check whether the TC value is exactly equal to 100, and the program will lock in a red box below if yes.

```
338  void MN_CtDemoCase14(void)
339  {
340    uint32_t wToggleCnt = 0;    //Indicates the I/O toggle count.
341
342    //Let P1.0 be the CAP0 counter's input source.
343    SN_GPIO1->MODE = 0x01;  //Set P1.0 as output mode.
344    SN_GPIO1->DATA = 0x00;  //Set P1.0 as output low.
345
346    CT16B0_Init();
347
348    //Set MR0 match value as 100.
349    SN_CT16B0->MR0 = 100;
350
351    //Set Counter Control as rising edge counter mode.
352    SN_CT16B0->CNTCTRL = (mskCT16_CTM_CNTER_RISING|mskCT16_CIS);
353
354    //Enable CAP0 function.
355    SN_CT16B0->CAPCTRL = mskCT16_CAP0EN_EN;
356
357    //Enable MR0 match interrupt.
358    SN_CT16B0->MCTRL = mskCT16_MR0IE_EN;
359
360    //Set CT16B1 to reset TC value.
361    SN_CT16B0->TMRCTRL = mskCT16_CRST;
362
363    //Wait until timer reset done.
364    while (SN_CT16B0->TMRCTRL & mskCT16_CRST);
365
366    //Let TC start CAP0 Counter function.
367    SN_CT16B0->TMRCTRL |= mskCT16_CEN_EN;
368
369    //Enable CT16B1's NVIC interrupt.
370    CT16B0_NvicEnable();
371
372    //Let P1.0 toggle 100 times to generate 100 times of rising edge.
373    for (wToggleCnt = 0; wToggleCnt < 100; wToggleCnt++)
374    {
375      SN_GPIO1->BSET = 0x01;      //Set P1.0 as output high.
376      UT_DelayNx10us(1);          //Delay 10us
377      SN_GPIO1->BCLR = 0x01;      //Set P1.0 as output low.
378      UT_DelayNx10us(1);          //Delay 10us
379    }
380
381    // Check whether TC matches MR0
382    if (iwCT16B0_IrqEvent == mskCT16_MR0IF)
383    {
384      iwCT16B0_IrqEvent = 0;  //Clear MR0 match interrupt variable.
385      //Check whether wToggleCnt is equal to TC value.
386      if (SN_CT16B0->TC == wToggleCnt)
387        while (1);                //Pass point
388    }
389    while (1);                    //Fail point
390  }
```

4.19 Demo Case 15

This example demonstrates CT16B0 input Capture function. Users should connect CT16B1_PWM0 to CT16B0_CAP0, the result is shown in Section 5.16.

Firstly, enable CT16B1 PWM0 function, set MR0 match value to (12000-1), and enable External Match Toggle to make PWM0 output the 1KHz square wave.

Secondly, enable CT16B0 CAP0 capture function, set counter mode and enable Rising Edge Capture/Falling Edge Capture/CAP0 Interrupt/CAP0 input function, and then reset the Timer. After the reset completes, FW enables TC count and waits for CAP0 capture.

Thirdly, start CT16B1 PWM0 output.

Fourthly, check CAP0 interrupt trigger signal.

Since the CAP0 internal count is not synchronized with the PWM0 output, so ignores the first retrieved CAP0 value, and then begins to retrieve CAP0 value stored in the buffer. This example retrieves a total of 20 results. After the completion of 20 times, start checking whether the existence of each data buffer is equal to 12000 (MR0 value + 1) or not, the program will lock in a red box below if yes.

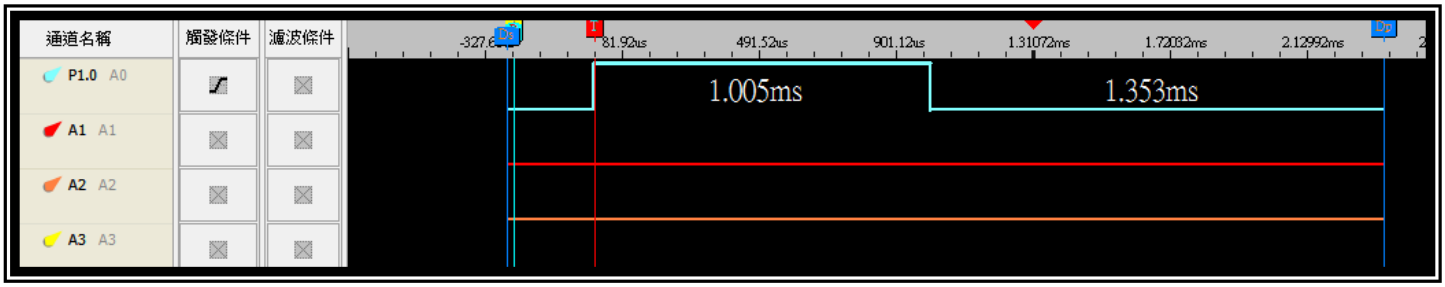
```
void MN_CtDemoCase15(void)
{
  uint32_t wCnt = 0;        //Indicates capture count times.
  uint16_t wPreCap0 = 0;    //Indicates the previous  CAP0 value.
  uint16_t wCap0Buffer[20]; //Used to save delta CAP0 values of 20 times.

  /***** Step 1: Setup the 1KHz toggle in CT16B1. *****/
  CT16B1_Init();

  //Set MR0 match value for 1KHz toggle rate output by External Match toggle method.
  //Thus, the match value should be (12000 - 1)@HCLK = 12MHz.
  SN_CT16B1->MR0 = 12000 - 1;
  //Set External Match for EMC0 toggle.
  SN_CT16B1->EMC = mskCT16_EMC0_TOGGLE;
  //Enable PWM0 External Match and IO.
  SN_CT16B1->PWMCTRL = (mskCT16_PWM0EN_EM0);
  SN_CT16B1->PWMIOENB = (mskCT16_PWM0IOEN_EN);
  //Set MR0 match to reset TC.
  SN_CT16B1->MCTRL = mskCT16_MR0RST_EN;
  //Set CT16B1 as the up-counting mode.
  SN_CT16B1->TMRCTRL = (mskCT16_CRST|mskCT16_CM_EDGE_UP);

  /***** Step 2: Setup CAP0 timer capture in CT16B0.*****/
  CT16B0_Init();
  //Set Counter Control as timer counter mode.
  SN_CT16B0->CNTCTRL = (mskCT16_CTM_TIMER|mskCT16_CIS);

  //Enable CAP0 function.
  SN_CT16B0->CAPCTRL =
      (mskCT16_CAP0RE_EN)|  //Enable rising edge capture.
      (mskCT16_CAP0FE_EN)|  //Enable falling dge capture.
      (mskCT16_CAP0IE_EN)|  //Enable CAP0 interrupt.
      (mskCT16_CAP0EN_EN);  //Enable CAP0 input function.

  //Set CT16B0 to reset TC value.
  SN_CT16B0->TMRCTRL = mskCT16_CRST;

  //Wait until timer reset done.
  while (SN_CT16B0->TMRCTRL & mskCT16_CRST);

  //Let TC start CAP0 Counter function.
  SN_CT16B0->TMRCTRL |= mskCT16_CEN_EN;

  //Enable CT16B0's NVIC interrupt.
  //CT16B0_NvicEnable();

  /***** Step 3: Start to toggle 1KHz from CT16B1 PWM0. *****/
  //Wait until timer reset done.
  while (SN_CT16B1->TMRCTRL & mskCT16_CRST);

  //Let TC start PWM function.
  SN_CT16B1->TMRCTRL |= mskCT16_CEN_EN;

```

```
457
458   /***** Step 4: Let CT16B0 CAP0 capture for 20 times *****/
459   //Ignore the first time of CAP0 capture but save it first.
460   while (1)
461   {
462     if (SN_CT16B0->RIS & mskCT16_CAP0IF)//Check whether CAP0 capture has done
463     {
464         SN_CT16B0->IC = mskCT16_CAP0IC; //Clear CAP0 status.
465         wPreCap0 = SN_CT16B0->CAP0;     //Save the CAP0 value.
466         break;
467     }
468   }
469
470   //Keep retrieving the delta value of each CAP0 status occurs for 20 times.
471   while (1)
472   {
473     if (SN_CT16B0->RIS & mskCT16_CAP0IF)//Check whether CAP0 capture has done
474     {
475         wCap0Buffer[wCnt] = SN_CT16B0->CAP0 - wPreCap0; //Save the delta value into
476         wPreCap0 = SN_CT16B0->CAP0;     //Save the CAP0 value.
477         SN_CT16B0->IC = mskCT16_CAP0IC; //Clear CAP0 status.
478         wCnt++;                         //Increase loop count.
479         if (wCnt == 20)                 //Check if already retrieving 20 times
480           break;
481     }
482   }
483   //Check whether all CAP0 delta in wCap0Buffer matches 12000.
484   for (wCnt = 0; wCnt < 20; wCnt++)
485   {
486     if (wCap0Buffer[wCnt] != 12000)//Check if matches 12000.
487       while (1);                   //Fail point
488   }
489   while (1);                       //Pass point
490 }
491
```

5 RESULT

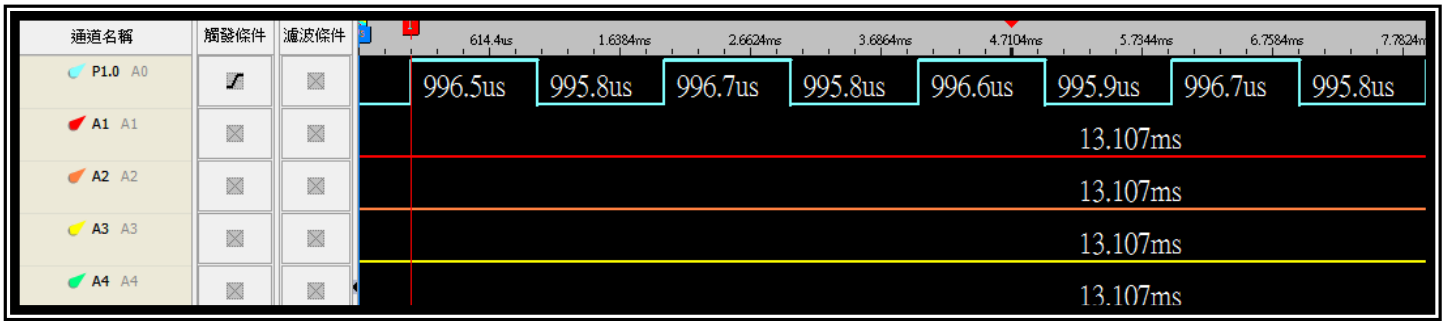
5.1 Result of Demo Case 0

This example is implemented with CT16B0. The TC count will match MR0 after about 1ms and stop counting. Users can observe P1.0 waveform which is an approximate 1ms pulse below.



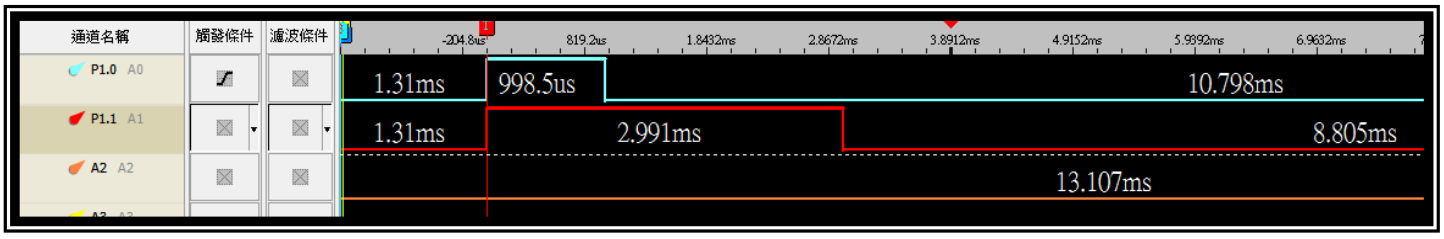
5.2 Result of Demo Case 1

This example is implemented with CT16B1. The TC count will match MR0 after about 1ms and then recount. Users can observe that P1.0 toggles with an approximately 500Hz frequency below.



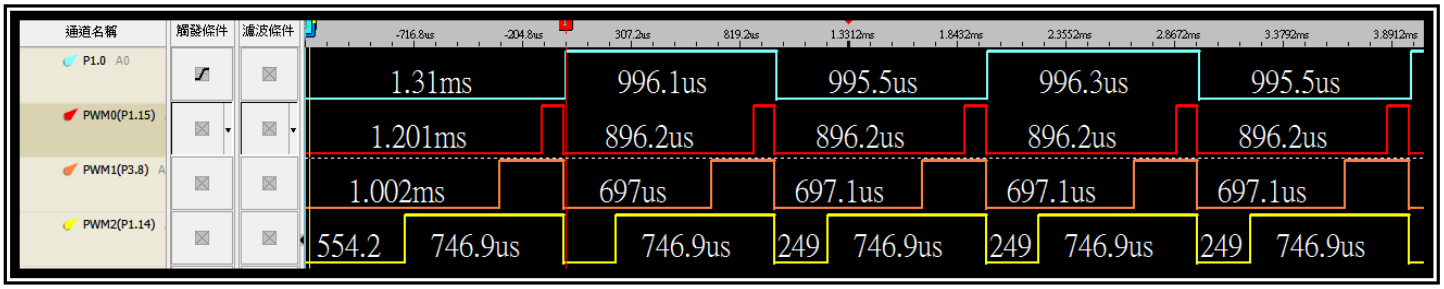
5.3 Result of Demo Case 2

This example is implemented with CT16B2. The MR2 match interrupt is triggered when the TC count matches MR2, and stop TC counting when the TC count matches MR3. Users can observe P1.0 waveform which is an approximate 1ms pulse, and P1.0 waveform which is an approximate 3ms pulse below.



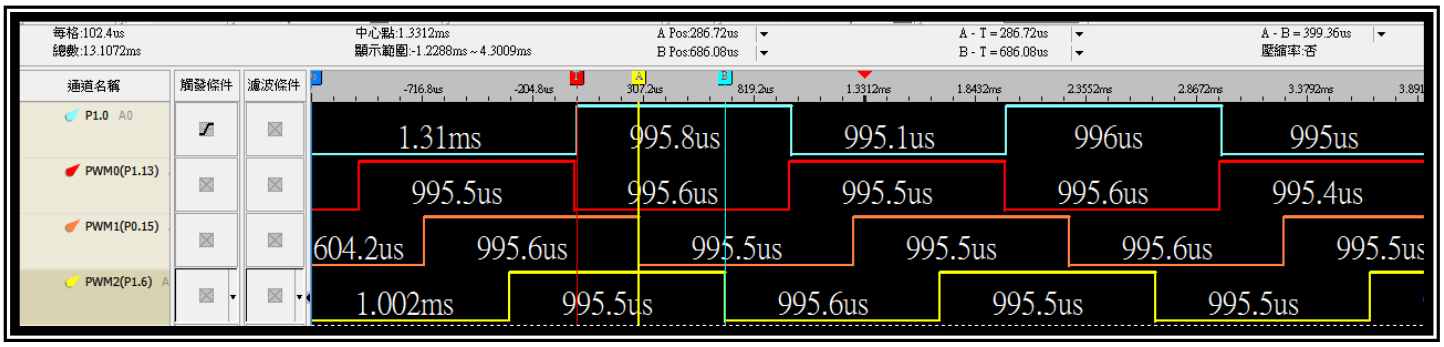
5.4 Result of Demo Case 3

This example demonstrates CT32B0 PWM function. The TC count will match MR3 every 1ms, and TC will recount to be the PWM cycle time, and make CT32B0_PWM0/PWM1/PWM2 output about 10%/30%/75% duty respectively. Users can observe P1.0 to know when MR3 matches the TC count in time (PWM period) below.



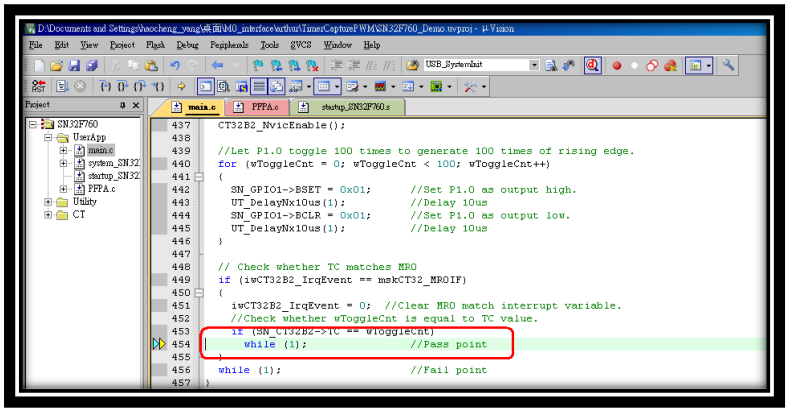
5.5 Result of Demo Case 4

This example demonstrates how to use CT32B1 PWM function with External Match every 1ms, and TC will recount to be the PWM cycle time, while CT32B1_PWM0/PWM1/PWM2 output about 0%/30%/70% duty respectively. Users can observe P1.0 toggle to know when MR3 matches the TC count in time (PWM period) below.



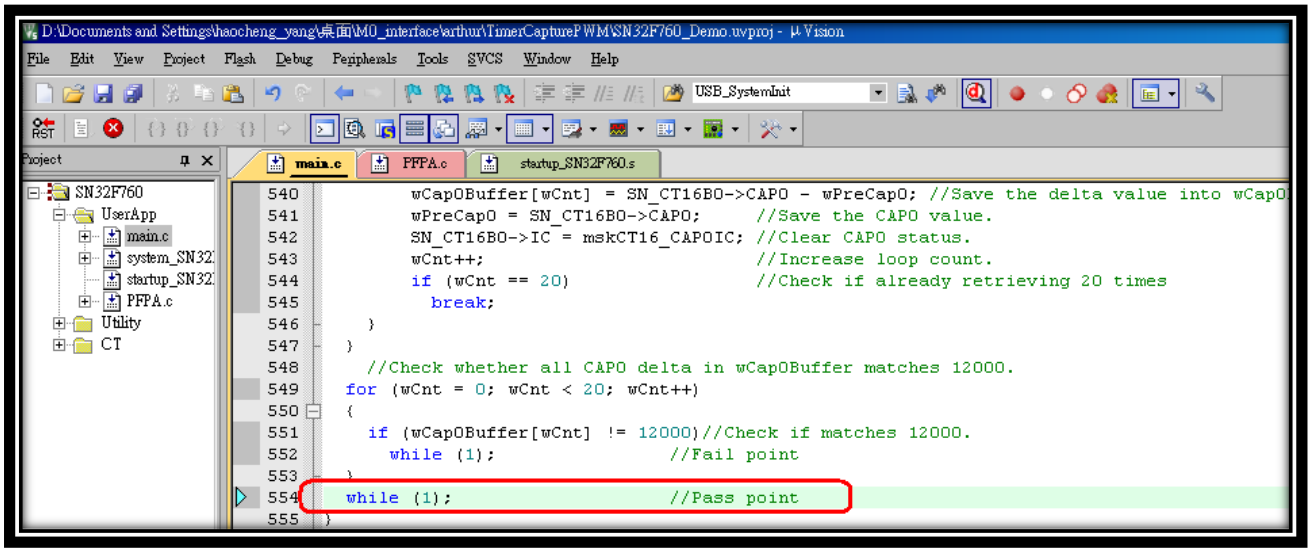
5.6 Result of Demo Case 5

This example demonstrates CT32B2 Counter input counting function. Users should connect P1.0 to CT32B2_CAP0. The P1.0 is programmed to output the waveform which is the input source of CT32B2_CAP0.



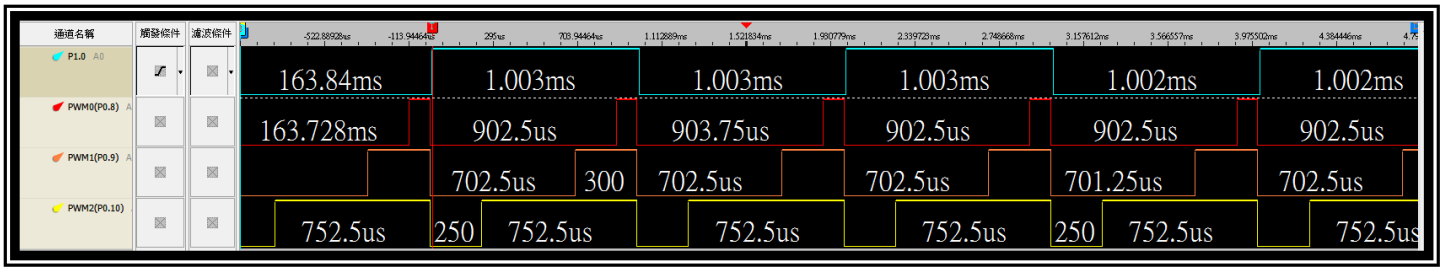
5.7 Result of Demo Case 6

This example demonstrates CT16B0 input Capture function. Users should connect CT32B0_PWM0 to CT16B0_CAP0, the result is shown below.



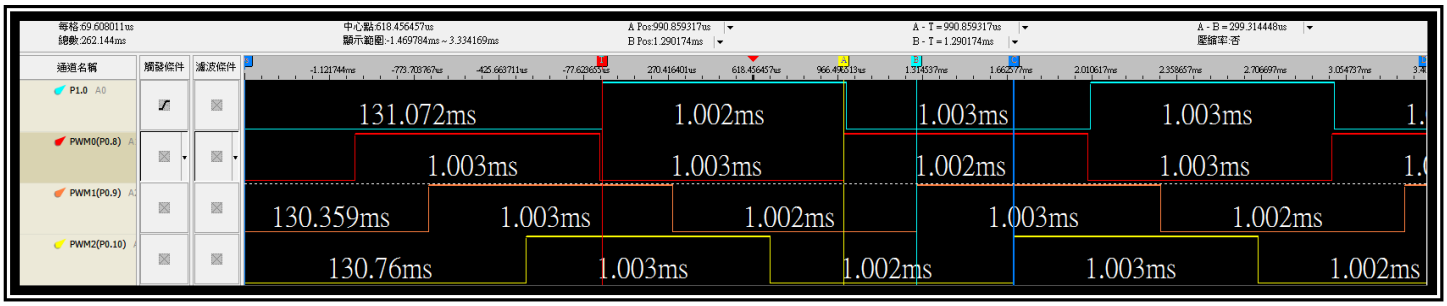
5.8 Result of Demo Case 7

This example demonstrates CT16B0 PWM function. The TC count will match MR9 every 1ms, and TC will recount to be the PWM cycle time, and make CT16B0_PWM0/PWM1/PWM2 output about 10%/30%/75% duty respectively. Users can observe P1.0 toggle to know when MR3 matches the TC count in time (PWM period) below.



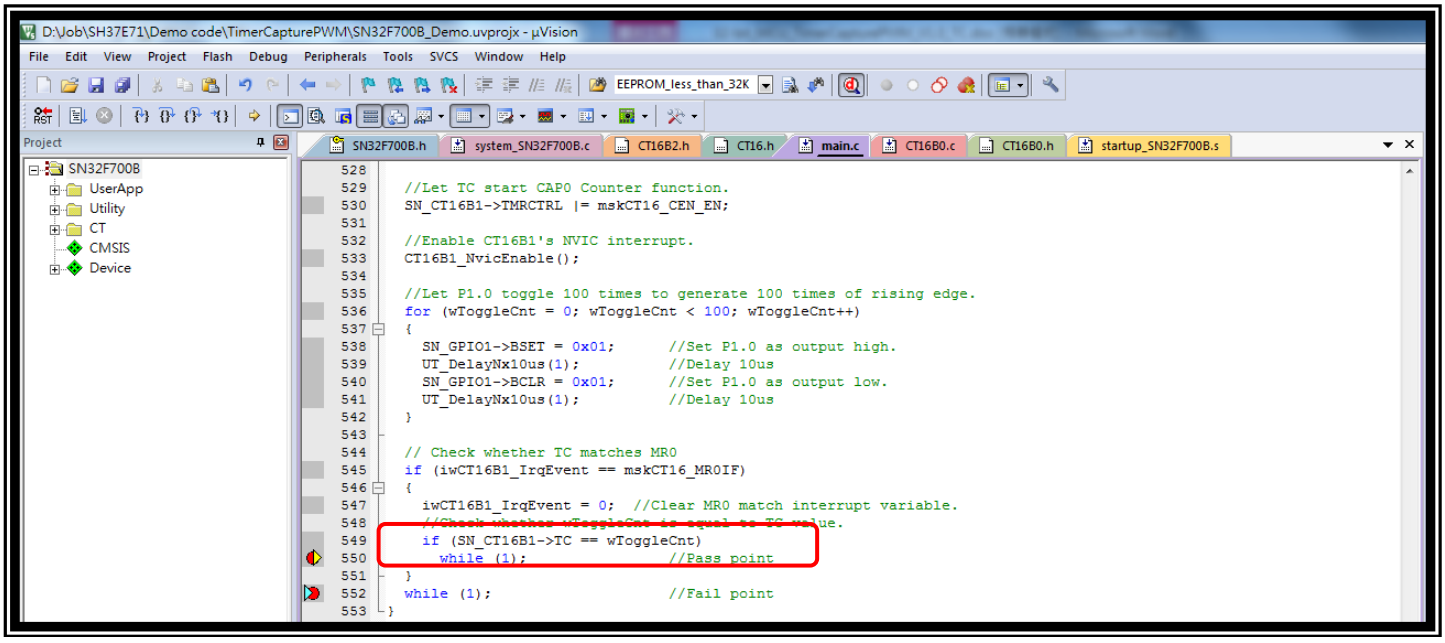
5.9 Result of Demo Case 8

This example demonstrates how to use CT16B0 PWM function with External Match. The TC matches MR9 every 1ms, and TC will recount to be the PWM cycle time, while CT16B0_PWM0/ PWM1/PWM2 output about 0%/30%/70% duty respectively. Users can observe P1.0 toggle to know when MR9 matches the TC count in time (PWM period) below



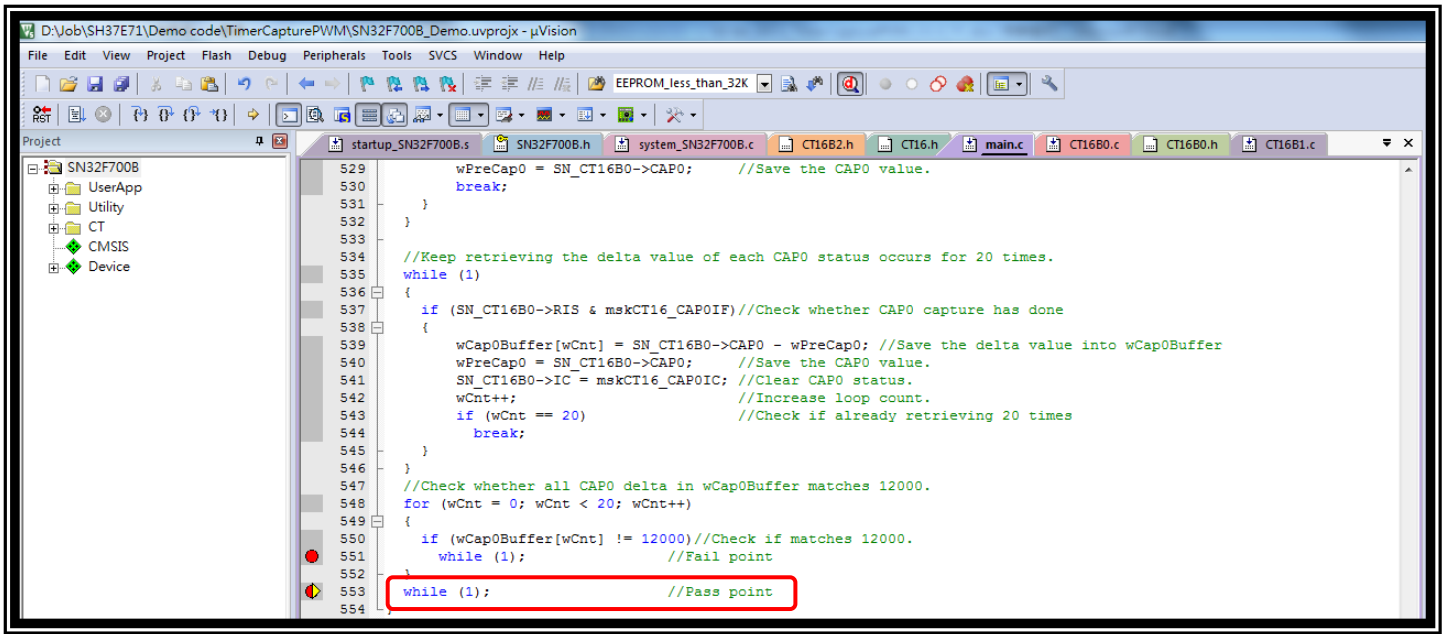
5.10 Result of Demo Case 9

This example demonstrates CT16B1 Counter input counting function.



5.11 Result of Demo Case 10

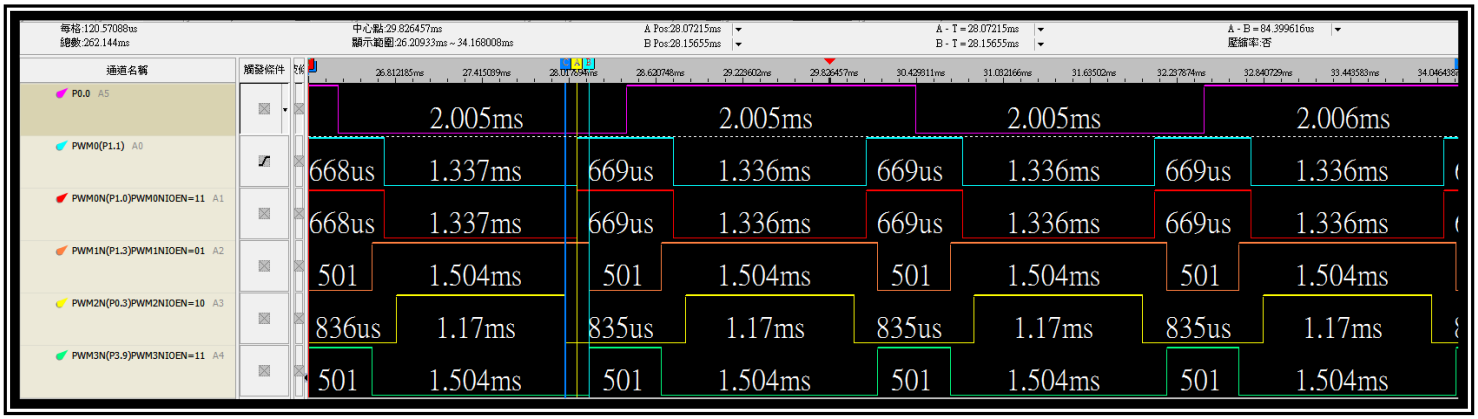
This example demonstrates CT16B0 input Capture function.



5.12 Result of Demo Case 11

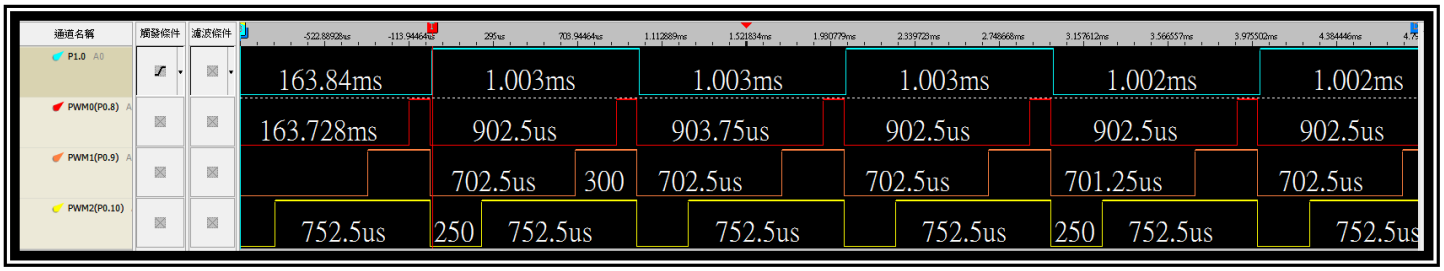
This example demonstrates CT16B2 PWM Dead-band function. The TC count will match MR9 every 2ms, and TC will recount to be the PWM cycle time. Users can observe P0.0 toggle to know when MR9 matches the TC count in time (PWM period) below.

PWM0N pin outputs the same signal without dead-band of PWM0. PWM1N pin outputs the inverse signal with dead-band of PWM1,but same High signal during dead-band period. PWM2N pin outputs the inverse signal with dead-band of PWM2, but same Low signal during dead-band period. PWM3N pin outputs the same signal with dead-band of PWM3.



5.13 Result of Demo Case 12

This example demonstrates CT16B1 PWM function. The TC count will match MR23 every 1ms, and TC will recount to be the PWM cycle time, and make CT16B1_PWM0/PWM1/PWM2 output about 10%/30%/75% duty respectively. Users can observe P1.0 toggle to know when MR23 matches the TC count in time (PWM period) below.

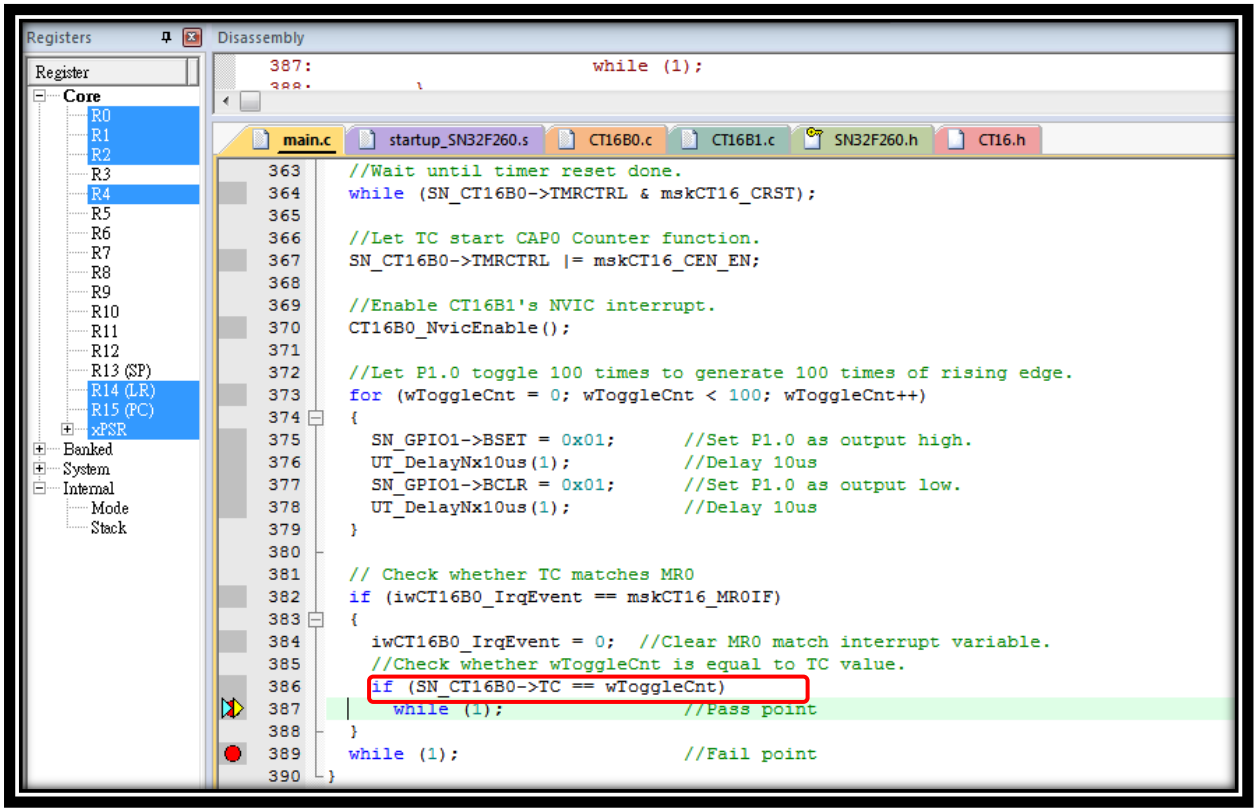


5.14 Result of Demo Case 13

This example demonstrates how to use CT16B1 PWM function with External Match every 1ms, and TC will recount to be the PWM cycle time, while CT16B1_PWM0/PWM1/PWM2 output about 0%/30%/70% duty respectively. Users can observe P1.0 toggle to know when MR23 matches the TC count in time (PWM period) below.

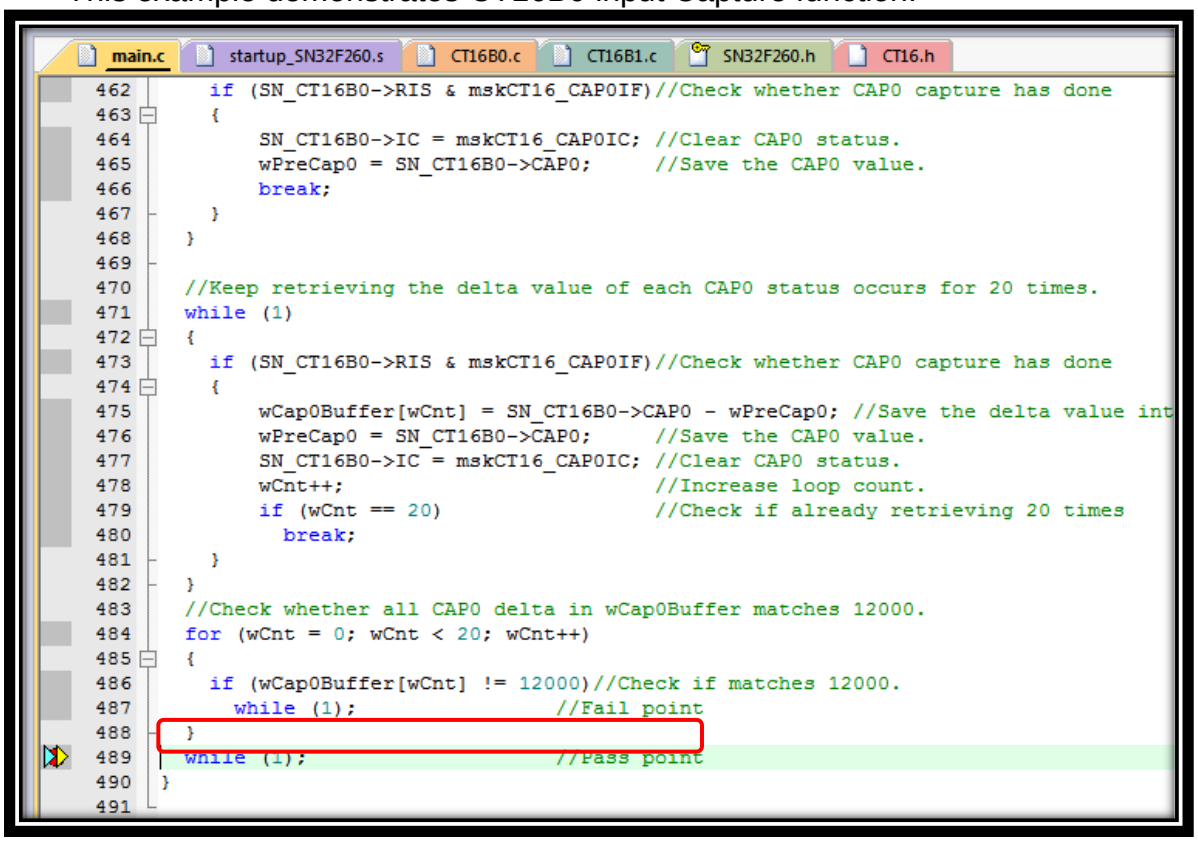
5.15 Result of Demo Case 14

This example demonstrates CT16B0 Counter input counting function.



5.16 Result of Demo Case 15

This example demonstrates CT16B0 input Capture function.



Main Office:
Address: 10F-1, NO. 36, Taiyuan Street, Chupei City, Hsinchu, Taiwan R.O.C.
Tel: 886-3-5600 888
Fax: 886-3-5600 889
Taipei Office:
Address: 15F-2, NO. 171, Song Ted Road, Taipei, Taiwan R.O.C.
Tel: 886-2-2759 1980
Fax: 886-2-2759 8180
Hong Kong Office:
Unit No.705, Level 7 Tower 1,Grand Central Plaza 138 Shatin Rural Committee Road, Shatin, New Territories, Hong Kong.
Tel: 852-2723-8086
Fax: 852-2723-9179
Technical Support by Email:

Sn8fae@sonix.com.tw